

(Note: Highlighted Text = Revisions Made)

1. Introduction Gus

1.1 Purpose

The purpose of this document is to outline the data flow, architecture, and components of the banking website. This website will be made for everyone to use, ranging from everyday people to businesses. This document provides the specifications for the development process so that it will meet the functional and nonfunctional requirements listed in the SRS.

1.2 Scope

The banking website will provide a web-based platform for users to create and manage their wallets and accounts. The website will enable users to transfer money, view their balance, and check their transaction history. It will also provide a secure login and banking experience.

1.3 Definitions, Acronyms, and Abbreviations

- **User Roles:** Customers, Employees, Administrators
- **Money Transfers:** The movement of money between accounts
- **Transaction:** To buy or sell something
- **SRS (Software Requirement Specification):** Document that outlines the requirements of a software product
- **SDD (Software Design Document):** Document that describes the system architecture, component design, and data flow of a software product

2. System Architecture Lance

2.1 Architectural Design

The banking system will be a web-based application where user transactions will occur on an online website. Therefore, the ideal architectural design model would primarily be based on a three-tier architecture, consisting of the Frontend, where the user interfaces with the website; the Backend, where data is processed; and the Database, where data is stored.

2.2 System Components

- User Interface / Frontend:
 - Tools used to construct the user interface:
 - JavaScript
 - CSS
 - HTML
 - User Interface Considerations:
 - Easy for users to efficiently navigate
 - Interface graphics remain consistent throughout navigation
 - Loading a page occurs within seconds during user navigation
- Data Processing / Backend:
 - Tool(s) used for Server-Side Logic:
 - nodeJS
 - JavaScript
- Data Storage / Database:
 - Main Database:
 - MySQL (or MongoDB)
 - Schema designs will be based around:
 - Users
 - Employee
 - Admin

- User

- Example:

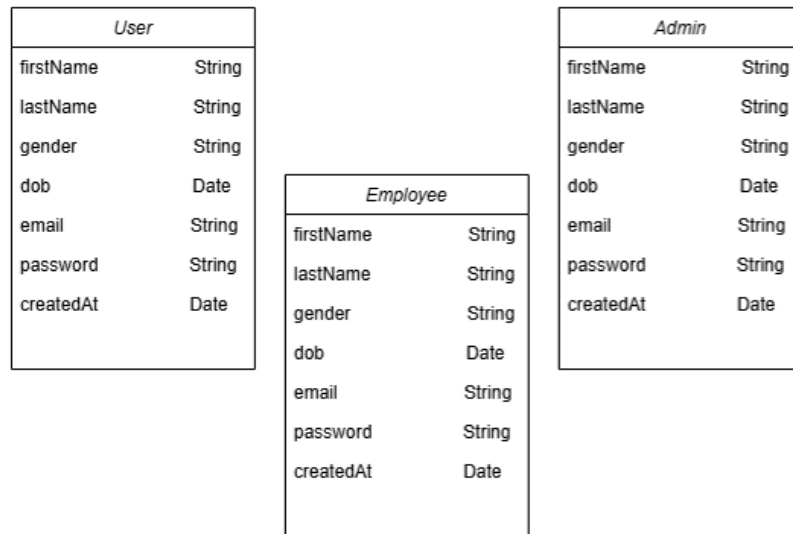


Figure 2.2-1: Users Database Schema

- Transactions

- Transaction
- Wallet

- Example:

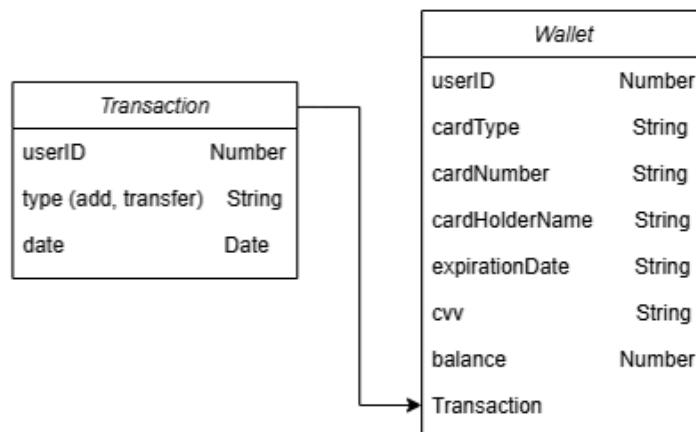


Figure 2.2-2: Transactions Database Schema

- (Possible) Third-Party Integrations:

- Authentication:
 - Auth0 / Duo Security
- Graphic Visualization:

■ Matplotlib / Tableau API

2.3 Data Flow Diagram (DFD)

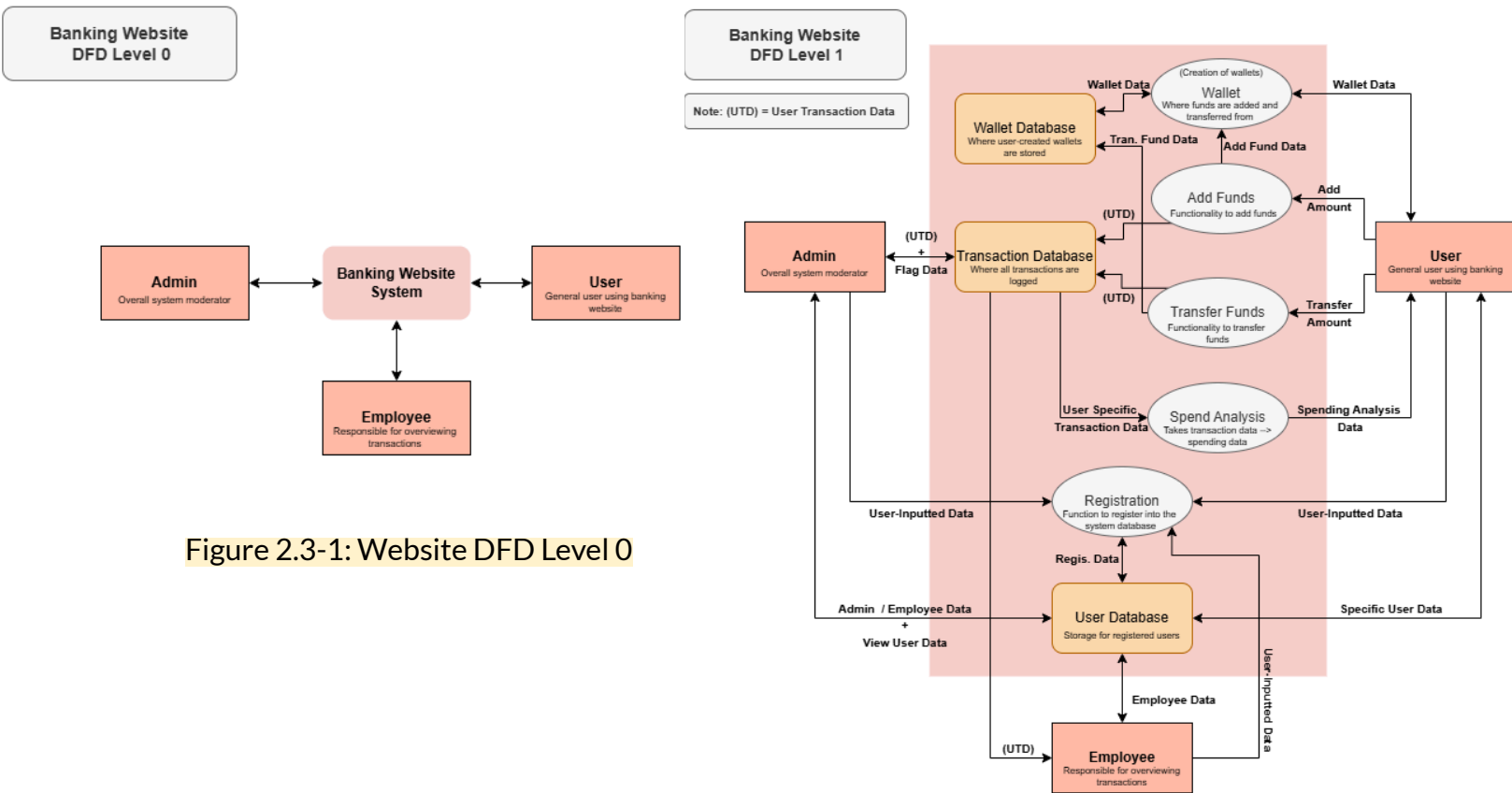


Figure 2.3-1: Website DFD Level 0

Figure 2.3-2: Website DFD Level 1

3. Design Considerations Anthony

3.1 Assumptions and Dependencies

The system shall require these external factors:

- Internet Access: Users must have a stable internet connection to access the banking website.

- Database Management System: The system shall use a database such as MySQL or MongoDB to store user account information, transaction history, and authentication details.
- Authentication Services: The system shall integrate with third party authentication providers (e.g., Auth0, Google Firebase Authentication, Duo Security) for two-factor authentication (2FA).
- Visualization Tools: Financial analytics shall rely on third party libraries or services such as Matplotlib or Tableau API.

3.2 Constraints

The system is subject to the following limitations and constraints:

- Web Based Interface: The system shall operate through a web browser
- User Load Capacity: The system shall support 100 concurrent users without performance degradation.
- Password Requirements: Users shall create strong passwords meeting the following criteria:
 - Minimum 8 characters
 - At least one numeric digit (0-9)
 - At least one lowercase letter (a-z)
 - At least one uppercase letter (A-Z)
 - At least one special character (!, @, #, \$, etc.)
- Fund Transfer Limitations: Users shall not transfer more funds than their available balance.

3.3 Security Considerations

To ensure system security, the following mechanisms will be implemented:

- Authentication:

- Users shall log in using secure credentials.
 - Two factor authentication (2FA) shall be required for additional security.
 - Authorization:
 - Different dashboards shall be provided for customers, employees, and administrators, each with distinct permissions.
 - Employees shall be able to view transaction overviews but cannot modify customer accounts.
 - Administrators shall be able to view, manage, and flag transactions.
 - Data Encryption:
 - The system will use HTTPS for secure communication between clients and servers.
 - Sensitive data, including passwords and financial transactions, shall be hashed before storage.
-

4. Detailed Design Lernik

4.1 User Registration and Authentication

1) **Sign-Up Flow:** The `signup.js` file collects user inputs (first name, last name, gender, DOB, email, password) from the sign-up form and validates them ensuring the password meets complexity requirements and that no digits appear in name fields. When the form is submitted, it sends a JSON payload to the backend endpoint (`/api/auth/signup`).

2) **Backend Processing:**

- Receives the sign-up request and first checks if the provided email is already registered.
- It generates a unique 9-digit account ID and creates a new user record in the database (using the User model).
- After storing the user, a JSON Web Token (JWT) is created—embedding the account ID, email, and user role—and then returned to the client.

3) Login Flow and Session Management:

- login.js handles the login form submission by sending the credentials to the appropriate endpoint (e.g., /api/auth/login for regular users, or alternative endpoints for admins and employees). And upon successful authentication (verified by authController.js), the JWT is generated and returned to the client.
- **Session Security:** The JWT token is stored in the client's sessionStorage, ensuring that the session persists only during the browser session. Also, each subsequent API request uses this token for user verification, helping to maintain session security and prevent unauthorized access.

4.2 Core Functionality

1. User interactions

- Deposit and Withdrawal
 - Deposit and withdrawal flows are handled by deposit.js and withdrawal.js.
 - Users choose between “Bank Users” and “Out of Bank Users” to reveal corresponding forms.

- These interactions include dynamically showing/hiding form sections and validating user inputs (for instance, ensuring that a user does not deposit or withdraw money to/from their own account).

2. Data processing

- Transaction Flow:
 - For deposits, after form submission in `deposit.js`, the system fetches the user's wallet details (via a call to `/api/wallet`) and then makes a POST request to create a deposit request (for either bank or external users).
 - For withdrawals, `withdrawal.js` similarly validates the input, verifies wallet balance, and then calls the withdrawal endpoint to process the transaction.
 - `incomingDeposits.js` consolidates incoming deposit and withdrawal requests, allowing users to confirm or reject pending transactions.

3. Error handling

- Throughout the client-side code (in `deposit.js`, `withdrawal.js`, and `login.js`), error handling is implemented using try-catch blocks to manage failed API calls.
- Error messages are both logged to the console and displayed to users through dedicated UI elements

- Input validations (such as checking that numeric fields contain only digits and that values are within acceptable ranges) further prevent erroneous data from being processed

4.3 Notifications (if applicable)

1) Password Reset:

- The `authController.js` simulates sending a password reset verification code. For testing, a fixed sample code (e.g., 123456) is used instead of sending a real email.

2) In-App Notifications:

- The `incomingDeposits.js` file manages notifications for pending deposit and withdrawal requests. Transaction status updates and confirmations are displayed via modals, badge updates, and on-screen messages.

4.4 Search and Filtering

1) Search Functionality:

- Admin and Employee Dashboards:
 - In `admin.html` and `employees.html`, search bars allow administrators and employees to search for users by account ID, with the corresponding JavaScript (in `admin.js` or `employees.js`) processing the input and dynamically filtering the registered users table.
- Transaction History Filtering:

- The incomingDeposits.js module includes logic to filter transaction records based on criteria such as transaction type (incoming vs. outgoing) and status. This filtering is applied after fetching a complete transaction history from the backend, ensuring that only relevant transactions are displayed.

5. External Interfaces Anthony

5.1 User Interfaces

The banking system provides distinct user interfaces based on roles:

Customer Dashboard

- Secure login with two-factor authentication (2FA).
- Account overview displaying wallet balances.
- Transaction history with filtering options.
- Spending analytics with graphical insights.
- Money transfer functionality between wallets and to other users.
- Account settings for updating personal details.

Employee Dashboard

- Secure login with 2FA.
- Access to customer transaction overviews.
- Ability to view flagged transactions for potential fraud detection.
- No permission to modify customer account details.

Administrator Dashboard

- Secure login with 2FA.
- Access to all system transactions and reports.
- Ability to flag, investigate, and manage suspicious activities.
- Permissions to oversee employee activities and set system policies.

5.2 API Interfaces

The system will expose the following API endpoints to facilitate banking operations:

Authentication Endpoints

- POST /api/auth/register – Registers a new user.
- POST /api/auth/login – Authenticates users and issues a session token.
- POST /api/auth/logout – Logs out the user and revokes the session.

User Management

- GET /api/users/{userId} – Retrieves user profile information.
- PUT /api/users/{userId} – Updates user account details.

Wallet & Transactions

- GET /api/wallets/{walletId} – Fetches wallet balance and details.
- POST /api/wallets/{walletId}/deposit – Adds funds to a wallet.
- POST /api/wallets/{walletId}/withdraw – Withdraws funds from a wallet.
- POST /api/transactions/transfer – Initiates a money transfer between wallets.
- GET /api/transactions/history – Retrieves a user's transaction history.

Spending Analytics

- GET /api/analytics/spending – Retrieves user spending trends.

Admin & Employee Controls

- GET /api/admin/transactions/flagged – Retrieves a list of flagged transactions.
- POST /api/admin/transactions/flag/{transactionId} – Flags a suspicious transaction for review.

5.3 Software Interfaces

The banking system relies on the following third-party software dependencies:

- Database Management: MySQL or MongoDB for storing user and transaction data.
- Authentication Services: Auth0, Google Firebase Authentication, or Duo Security for 2FA.
- Data Visualization: Matplotlib or Tableau API for displaying financial analytics.
- Security & Encryption: OpenSSL for encrypting sensitive data and bcrypt for hashing passwords.
- Web Communication: HTTPS for secure API communication.

6. Non-Functional Requirements Mapping Lernik

This Level 0 DFD below shows how the user's requests (e.g., sign-up, login, deposit, or withdrawal) flow into two main processes—Authentication and Transaction.

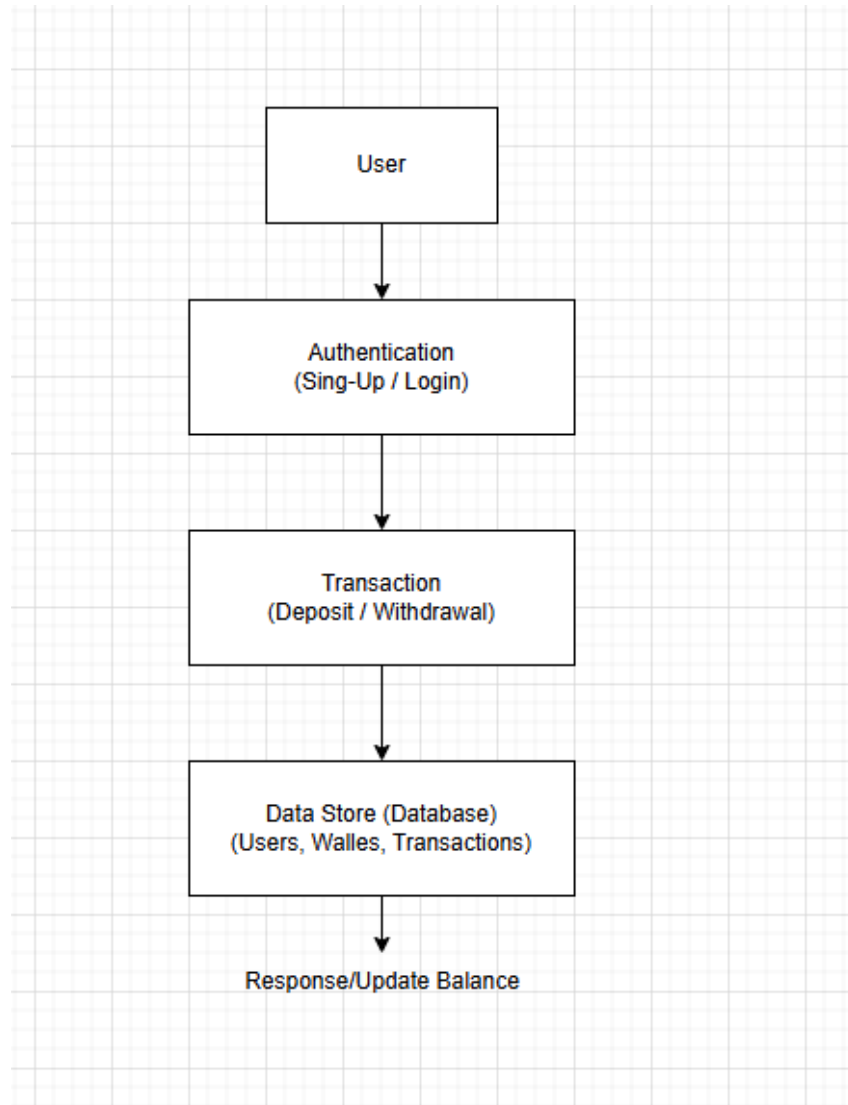


Figure 6-1: Level 0 User Request DFD

Level 1 – Authentication (Sign-Up / Login)

This diagram breaks down the authentication process into three main steps:

- **Validate Input:** The system first checks that all required user information (such as name, email, password, etc.) is provided and properly formatted.
- **User Record Verification/Creation:** Next, the system verifies if the user already exists in the database. For login, it confirms the credentials, and for sign-up, it creates a new user record.
- **Generate JWT:** Upon successful validation and verification/creation, a JSON Web Token (JWT) is generated containing the account ID, email, and user role. This token, along with user data, is then returned to the client to establish a secure session.

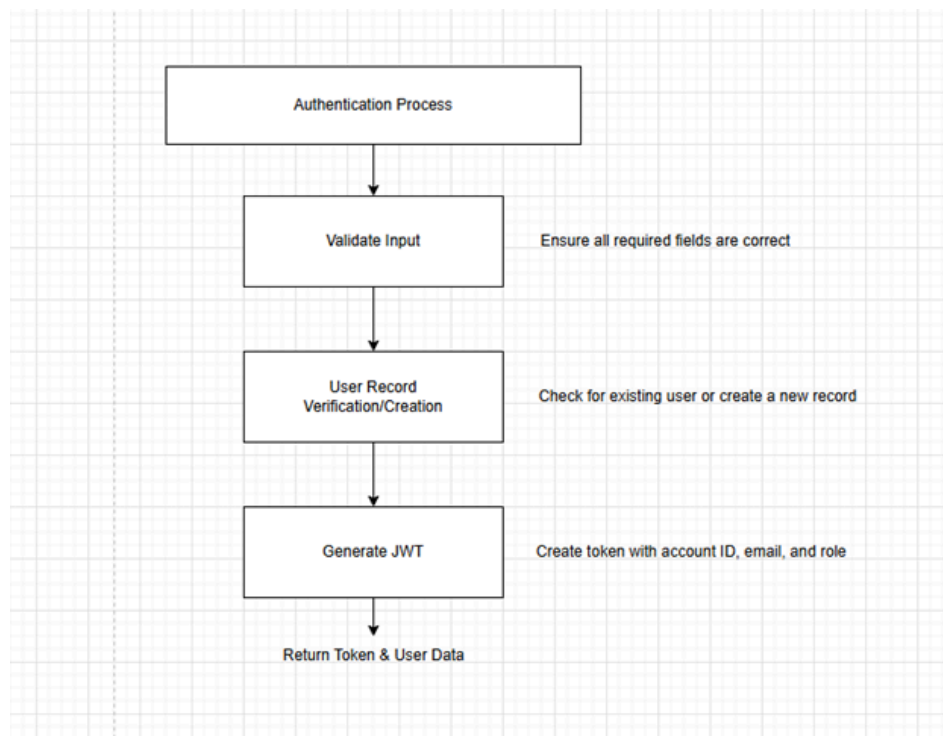


Figure 6-2: Level 1 Authentication DFD

Level 1 - Transaction Process (Deposit / Withdrawal)

This diagram breaks down the transaction process into three main steps:

Verify JWT & Permissions: The system checks that the user's authentication token (JWT) is valid and confirms the user's role and permissions before processing the transaction.

Validate Request: The system validates the transaction details, ensuring that the deposit or withdrawal request meets criteria such as correct account IDs, proper amounts, and sufficient wallet balances.

Update Database: Once the request is validated, the system updates the database by adjusting wallet balances and creating a new transaction record. The response, including updated balances or transaction confirmations, is then sent back to the user.

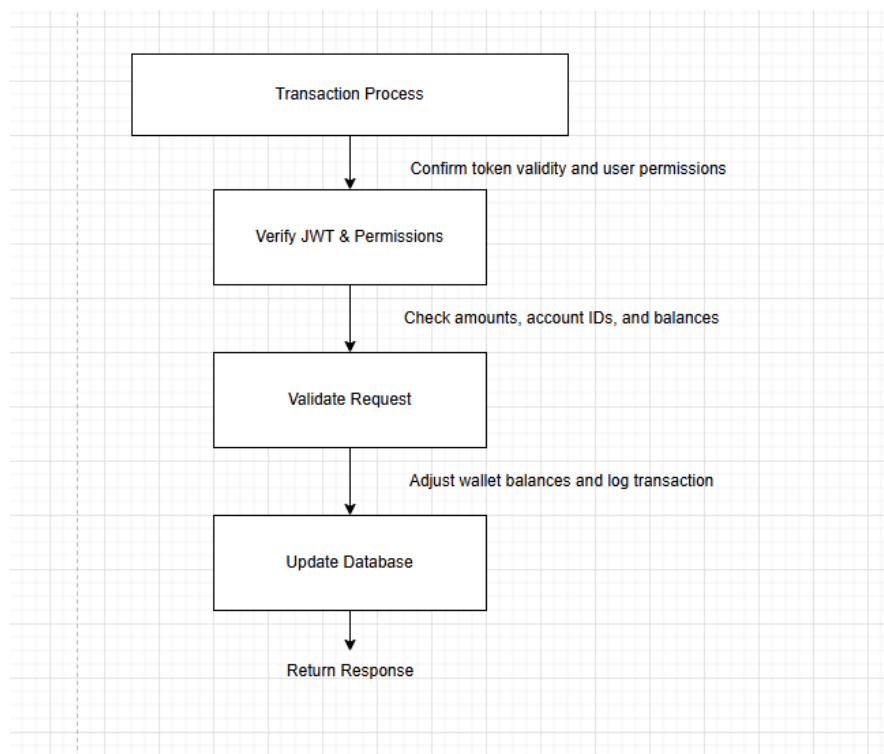


Figure 6-3: Level 1 Transaction Process DFD

7. Appendices Gus

7.1 Glossary

- **Balance:** A current total of money held within a bank account
- **Wallet:** A digital storage of different payment methods
- **HTTPS:** Hypertext Transfer Protocol Secure
- **API:** Application Programming Interface
- **SQL:** Structured Query Language

7.2 References

- **MongoDB:** a NoSQL database that is used for data storage and management
- **MySQL:** an SQL database that is used for data storage and management
- **Two Factor Authentication Services:** Auth0, Google Firebase Authentication, Duo Security
- **Graphic Visualization Services:** Matplotlib, Tableau API